



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #1
File Handling in C***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: File handling in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Data created during program execution is stored temporarily in variables. However, once the program ends, this data is lost. To preserve data even after the program terminates, **files** are used. File handling allows a C program to **create, read, write, and modify files** stored on disk. This is crucial for tasks such as **saving user input, processing large datasets, logging system activities, or reading configuration files**.

File handling enables the below tasks:

- **Persistent data storage** beyond the life of a program.
- **Data sharing** between programs.
- **Handling large volumes of data** efficiently (which might not fit in memory)

File handling allows programs to **create, open, read, write, append, and close files** stored on the computer's storage device. File handling gives us the tools(functions) to manage the different types of files efficiently.

Real-Life Examples

1. Saving User Data in a Program
2. Creating Logs for a System - Record program events, errors, or status updates
3. Storing Game Scores - Save high scores or game progress that can be retrieved later.
4. Reading Configuration Settings - Load program preferences from a file during startup.
5. Writing Results from a Scientific Simulation
6. Handling Financial Transactions - Record billing or transaction data securely in a file.
7. Maintaining Audit Trails - Keep track of user actions and system access for security reviews.

File Classification

Files are referred to as data files in C. There are basically two distinct types of data files available in C.

- Text Files
- Binary Files

Text Files

The simplest type of file is a **text file**, typically created with a **.txt extension** using any basic text editor. Internally, **it stores data as ASCII characters**, making the content **easily readable by humans**. However, text files are not secure, as the information is stored in plain text, and they tend to occupy more storage space compared to binary files.

Binary Files

A binary file stores data using the binary number system (0s and 1s) and typically has a **.bin extension**. Since it saves information in the same format as it's stored in computer memory, it uses **less storage space**. Unlike text files, binary files are **not human-readable**, making them a safer option for **storing sensitive or structured data**.

Major Operations on Files

- Creation of a new file or Opening an existing file
- Reading data from a file
- Writing data in a file
- Closing a file
- Rewinding a File Pointer
- Moving the File Pointer
- Getting the Current File Position
- Checking for End-of-File
- Removing a File
- Renaming a File

To perform any operation on Files, the **physical filename**, the **logical filename** and the **mode** must be connected using `fopen()`

- **Physical Name:** A file is maintained by the OS. The OS decides the naming convention of a file.
- **Logical Name:** In a C Program, identifier is used to refer to a file. Also called as FileHandle.
- **Mode:** Can be read only, write only, append or a combination of these.

1. Creation of a new file or Opening an existing file

To create a new file, one has to use a **built-in FILE structure**. Create a pointer to FILE type. This must hold a logical reference to physical file on disk (hard disk) using the `fopen()` function.

```
FILE *fp;  
fp = fopen("filepath", "mode");
```

fopen(): Opens a file in the specified mode and **returns a FILE pointer — a reference to a structure that holds information about the file's attributes**. If the file is opened successfully, this pointer is returned; otherwise, NULL is returned. **File opening can fail for various reasons, such as the file not existing or lacking the necessary permissions for the requested mode**. The first argument, `filePath`, is a pointer to a string representing the name of the file to be opened. The second argument, `mode`, defines the type of access (read, write, append, etc.). It's important to note that initializing a FILE pointer does not load the entire file into memory — additional file handling functions must be used to access or modify the file's content.

A file in C can be opened using **different modes based on the desired operation**.

- **"r"** – **Opens an existing file for reading only**. The file must already be present; otherwise, the operation fails.
- **"w"** – **Opens a file for only writing**. If the file exists, its contents are erased. If it doesn't, a new empty file is created.
- **"a"** – **Opens a file to append data at the end**. If the file does not exist, it is created automatically.
- **"r+"** – **Opens an existing file for both reading and writing**. The file must exist; otherwise, the operation fails.
- **"w+"** – **Creates a new file for both reading and writing**. If the file exists, it is overwritten.
- **"a+"** – **Opens a file for both reading and appending**. Data is always added to the end of the file. If the file doesn't exist, it is created.

fclose(): When a file is opened, certain data structures are created in memory to manage the file operations. Once the file is no longer needed, these **resources should be released by closing the file using the fclose() function**. This action **closes the stream, flushes any remaining data in the buffers to the file, and frees associated memory**. If the file is closed **successfully, fclose() returns 0; otherwise, it returns EOF to indicate an error**.

Consider, **data.txt** contains the below data.

welcome to file
handling
in C

Coding Example_1: Demo of fopen and fclose.

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r"); //file must be existing to open in read mode. Else returns NULL
    if(fp== NULL)
        printf("file cannot be opened\n");
    else {
        //perform reading here in else using any functions .then close the file
        printf("File can be accessed for reading\n");
        fclose(fp);
    }
    return 0;
}
```

```
C:\Users\Dell>a
File can be accessed for reading
```

2. Read/Write data from/to a file

This can be categorized into the following types:

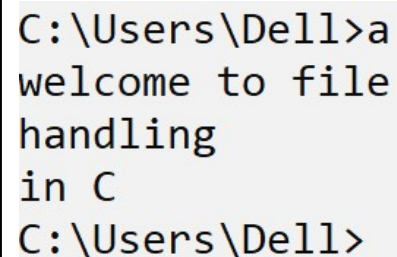
- **Character-based I/O** – Reading and writing one character at a time.
- **String-based I/O** – Handling entire lines or strings at once.
- **Formatted I/O** – Input and output using specific formats, similar to scanf() and printf().
- **Block I/O** – Reading from or writing to files in blocks of data, typically used for binary files.

Character based functions:

- **fputc(int ch, File_pointer):** Writes a character (an unsigned char) specified by the argument ch to the specified stream and advances the position indicator for the stream. If there are no errors, the same character that has been written is returned. If an error occurs, EOF is returned and the error indicator is set.
- **fgetc(file_pointer):** Reads a character from the file and returns the character read as an unsigned char cast to an int(ascii) and advances the file pointer to the next byte to be read or EOF on end of file **or error**.
- **getc(FILE *fp) and putc(char ch, FILE *fp):** The operation performed by getc() and putc() functions is the same as that performed by fgetc() and fputc() functions.

Coding Example_2: Demo of fgetc and fputc

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r"); // file must be existing
    if(fp== NULL)
        printf("file cannot be opened\n");
    else
    {
        char ch;
        while((ch=fgetc(fp)) != EOF)
        {
            //printf("character read is %c",ch);
            //printf("%c",ch);
            fputc(ch,stdout);// writing to standard output that is terminal
        }
        fclose(fp);
    }
    return 0;
}
```



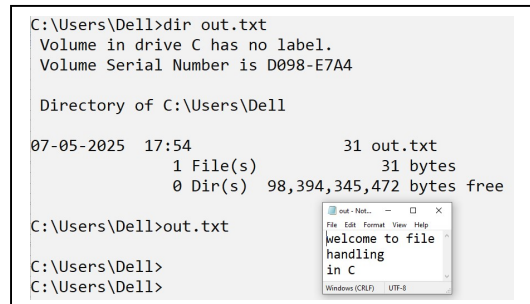
String based Functions:

- **fputs(const char *str, int n, FILE *fp):** Writes a string to the specified stream up to but not including the null character. Returns a non-negative value, or else on error it returns EOF.

- **fgets(char *str, FILE *fp):** Reads a line from the specified stream and stores it into the string pointed to by str. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first. On success, the function returns the same str. On error or if EOF is encountered, it returns NULL.

Coding Example_3: Demo of fgets and fputs. out.txt created with the contents of data.txt

```
#include<stdio.h>
int main(){
    FILE *fp1 = fopen("data.txt","r");
    FILE *fp2 = fopen("out.txt","w");
    char ch[500];
    if(fp1==NULL || fp2 ==NULL)
        printf("issue in opening the file\n");
    else {
        while(fgets(ch,500,fp1) != NULL) // change 500 to 2 and check the output
        {
            fputs(ch, fp2);
        }
        fclose(fp1); fclose(fp2);
    }
    return 0;
}
```



Formatted IO Functions:

When a data file is organized in a specific format—such as a line containing five integers separated by spaces—and the programmer knows this format in advance, it is more efficient to use formatted file I/O functions instead of reading character by character. These functions allow structured reading and writing based on expected data patterns. The fscanf() function works like scanf(), but it takes a file pointer as its first argument to read data directly from a file. fprintf is similar to printf().

- **fprintf(file_pointer,"format_string", arg1,arg2,...):** Sends formatted output to a stream. If successful, the total number of characters written is returned otherwise, a negative number is returned .



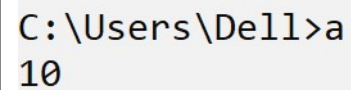
- **fscanf(file_pointer,"format_string",&arg1,&arg2,...):** Reads formatted input from a stream and returns the number of input items successfully matched and assigned, which can be fewer than provided for, or even zero in the event of an early matching failure.

Assume, Formatted.txt file contains the below data.

```
10    20    30
40    10    20
```

Coding Example_4: Demo of fscanf and fprintf to read an integer from a file and write to the terminal

```
#include<stdio.h>
int main(){
    FILE *fp1 = fopen("Formatted_data.txt","r");
    if(fp1==NULL)
        printf("issue in opening the file\n");
    else{
        int a;          fscanf(fp1,"%d",&a);
        fprintf(stdout,"%d",a); fclose(fp1);
    }
    return 0;
}
```

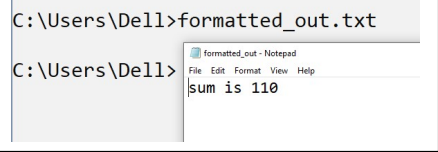


```
C:\Users\De11>a
10
```

Coding Example_5: Reading a set of integers from a file called numbers.txt and writing the sum of these integers to a new file.

Version 1: We know the number of integers to be read from the data file – n is hardcoded.

```
void read(int *a, int n,FILE *fp);
int find_sum(int *a, int n);
int main(){
    FILE *fp1 = fopen("formatted_data.txt","r");
    FILE *fp2 = fopen("formatted_out.txt","w");
    if(fp1==NULL || fp2==NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];          int n = 5;
        read(a,n,fp1);
        int sum = find_sum(a,n);
        //fprintf(stdout,"sum is %d",sum);
        fprintf(fp2,"sum is %d",sum);
        fclose(fp1);  fclose(fp2);
    }
    return 0;
}
```



```
C:\Users\De11>formatted_out.txt
```

```
formatted_out - Notepad
File Edit Format View Help
sum is 110
```

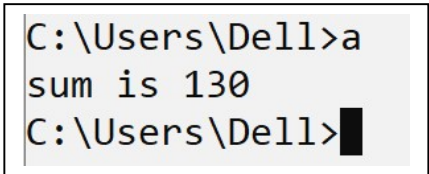
```
void read(int *a, int n, FILE *fp) {
    int i;
    for(i = 0; i < n; i++)
        { fscanf(fp, "%d", a+i); }
}
```

```
int find_sum(int *a, int n) {
    int i;
    int sum = 0;
    for(i = 0; i < n; i++)
        { sum = sum + *(a+i); }
    return sum;
}
```

Version 2: If we do not specify n in version 1, code fails. So use the return value of fscanf to find n

```
#include<stdio.h>
int read(int *a, FILE *fp);
int find_sum(int *a, int n);
int main(){
    FILE *fp1 = fopen("formatted_data.txt", "r");
    FILE *fp2 = fopen("formatted_out.txt", "w");
    if(fp1 == NULL || fp2 == NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];
        int n = read(a, fp1);
        int sum = find_sum(a, n); // same as version 1
        fprintf(stdout, "sum is %d", sum);
        fclose(fp1); fclose(fp2);
    }
    return 0;
}
```

```
int read(int *a, FILE *fp)
{
    int i = 0;
    while (fscanf(fp, "%d", a+i) != EOF)
        i++;
    return i;
}
```



```
C:\Users\De11>a
sum is 130
C:\Users\De11>
```

Block read / write operation on File

- **fwrite(const void *ptr, size_t size, size_t n, FILE *fp):** This function writes the data specified by the void pointer ptr to the file. **On success, it returns the count of the number of items successfully written to the file. On error, it returns a number less than n.**
- **fread(void *ptr, size_t size, size_t n, FILE *fp):** This function is commonly used to read binary data. It accepts the same arguments as the fwrite() function does. The function reads n items from the file where each item occupies the number of bytes specified in the second argument. **The function on success reads n items from the file and returns n and on error or end of the file, it returns a number less than n.**

Coding Example_6: Demo of fwrite and fread functions. Employee details to be read from the user and writing a data file.

```
#include<stdio.h>
#include<stdlib.h>
struct employee
{
    char name[15];    char designation[15];    int age;    float salary;
}emp;
int main()
{
    FILE *fp;
    fp = fopen("employee_data.txt", "w"); // at the end of the program execution, open and
    see the contents of the data file. Not in readable format
    if(fp == NULL)
        printf("Error opening file\n");
    else
    {
        char ch;
        do
        {
            printf("\nEnter the name:\t");
            scanf("%s",emp.name);
            printf("Enter the designation:\t");
            scanf("%s",emp.designation);
            printf("Enter the age : ");
            scanf("%d",&emp.age);
```

```
Enter the name: sindhu
Enter the designation: Asst.Professor
Enter the age : 30
Enter the salary : 9000000

Do you want to add another data (y/n) : y

Enter the name: indhu
Enter the designation: Asst.Prof
Enter the age : 80
Enter the salary : 8000000

Do you want to add another data (y/n) : n
```

NAME	DESIGNATION	AGE	SALARY
sindhu	Asst.Professor	30	9000000.000000
indhu	Asst.Prof	80	8000000.000000

```
        printf("Enter the salary : ");
        scanf("%f",&emp.salary);
        fwrite(&emp,sizeof(emp),1,fp);
        printf("\nDo you want to add another data (y/n) : ");
        fflush(stdin);
        ch = getchar();
    } while(ch=='y' || ch=='Y');
    fclose(fp);
    fp = fopen("employee_data.txt","r");
    if(fp == NULL)
        printf("cannot open the file\n");
    else
    {
        printf("\n\tNAME\tDESIGNATION\tAGE\tSALARY\n");
        while(fread(&emp,sizeof(emp),1,fp)>0)
            printf("\n\t%s\t%s\t%d\t%f\n",emp.name,emp.designation
            ,emp.age,emp.salary);
        fclose(fp);
    }
}
return 0;
}
```

Random Access to a file

You can jump to a specific position in the file and read/write data — without reading everything sequentially. Files permit non-sequential or random access to its contents using C functions. To access a file randomly, you open the file, seek a particular location, and read from or write to that file. In order to access a particular file or record C supports below functions.

fseek(), ftell(), rewind()

- **int fseek(FILE *stream, long offset, int whence);**

Used to set the file position indicator for the stream to a new position. This function accepts three arguments. The first argument is the FILE stream pointer returned by the fopen() function. The second argument ‘offset’ tells the amount of bytes to seek. The third argument ‘whence’ tells from where the seek of ‘offset’ number of bytes is to be done. The available values for whence are as below. Returns zero if successful, else it returns a non-zero value.

	Value	whence
SEEK_SET	0	Beginning of file
SEEK_CUR	1	Current position
SEEK_END	2	End of file

- **ftell(FILE *stream)**

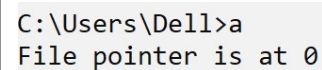
Returns the current file position of the given file_pointer. If an error occurs, -1 is returned.

- **rewind(FILE *stream)**

The function rewind sets the file_pointer position to the beginning of the file. Doesn't return any value.

Coding Example_7: Demo of ftell () . Pointer is at the beginning of the file just after opening the file in read mode

```
#include<stdio.h>
int main(){
    FILE *fp = fopen("data.txt","r"); // try w and a as well
    if(fp == NULL) printf("cannot open the file");
    else {
        printf("File pointer is at %d\n",ftell(fp));
        fclose(fp);
    }
    return 0;
}
```



```
C:\Users\Dell>a
File pointer is at 0
```

Coding Example_8: Demo of fseek()

```
#include<stdio.h>
int main()
{
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else
    {
        printf("Initially pointer is at %d\n",ftell(fp));
        fseek(fp,5,SEEK_SET); // start from the beginning
        printf("After fseek pointer is at %d\n",ftell(fp)); //5
        //fseek(fp,2,SEEK_SET);
        //printf("%d",ftell(fp)); // 2
    }
}
```

```

//fseek(fp,2,SEEK_CUR); //start from the current position of the pointer
//printf("%d",ftell(fp)); //7
fseek(fp,-9,SEEK_END); //start from the end of the file
printf("after seeking from end %d",ftell(fp));
printf("character just read is ");
putchar(fgetc(fp));
fclose(fp);
}
return 0;
}

```

```

C:\Users\Dell>a
Initially pointer is at 0
After fseek pointer is at 5
after seeking from end 22character just read is i

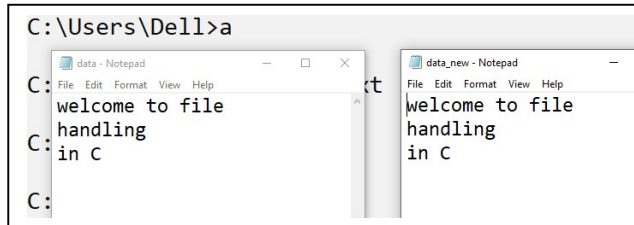
```

Coding Example_9: Copy the contents of one File to another

```

#include<stdio.h>
int main(){
    FILE *fp1 = fopen("data.txt","r");
    FILE *fp2 = fopen("data_new.txt","w");
    char ch;
    while((ch = fgetc(fp1)) != EOF)
        fputc(ch, fp2); // copy character by character from 5th position
    fclose(fp1); fclose(fp2);
    return 0;
}

```



Coding Example_10: Usage of rewind()

```

#include<stdio.h>
int main(){
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else{
        fseek(fp,-5,SEEK_END); printf("%d\n",ftell(fp));
        fseek(fp,0,SEEK_SET)//rewind(fp); to move the pointer to the beginning
of the file.
        printf("%d\n",ftell(fp)); fclose(fp);
        return 0;
    }
}

```

```

C:\Users\Dell>a
26
0

```

Handle Files, Handle Data – Happy Coding!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #2
Error handling in File Handling***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Error handling in File Handling**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Errors are the problems or the faults that occur in the program, which makes the behavior of the program abnormal, and experienced programmers and developers can also make these faults. **Programming errors** are also known as the **bugs or faults**.

When working with files in C, it's crucial to ensure that operations such as **opening, reading, writing, or closing a file are executed safely**. However, various factors such as **missing files, access restrictions, or disk issues can cause these operations to fail**. If such failures are not handled properly, they may lead to incorrect program behavior, crash, or data loss. The most common errors that may occur are:

1. **Reading beyond the end-of- file:** EOF is a condition in a computer operating system where no more data can be read from a data source. The data source is usually a file or stream.
2. **Performing operations on the file that has not still been opened:** Before performing any operation on a file, you must first open it. An attempt to perform operations on a file that has not been opened yet will lead to error
3. **Writing to a file that is opened in the read mode:** A mode is used to specify whether you want to open a file to perform operations like read, write append etc.
4. **Opening a file with invalid filename:** If you attempt to upload a file that has an invalid file name or type you will receive error.
5. **Write to a write-protected file:** Write-protected file is the ability to prevent new information from being written or old information being changed. In other words, information can be read, but nothing can be added or modified.

Error handling allows the program to **detect if a file operation failed, respond gracefully (e.g., display an error message, attempt recovery, or exit safely)** and **maintain robustness and user-friendliness**. It helps during the processing of input-output statements by catching severe errors that might not otherwise be noticed. In C, standard library functions provide return values and flags that can be checked to identify and manage such issues effectively.

Types of Error Handling in C

C does not have built-in exception handling features. Instead, **it uses manual error handling techniques**. These can be broadly categorized into:

1. **Return value checking (Most Common in C):** Most functions return a value that indicates whether they succeeded or failed. These values must be explicitly checked to detect errors and handle them properly. This helps avoid unexpected behavior or program crashes by ensuring that operations only proceed when they are safe. This practice also simplifies the debugging process by making it easier to identify where and why something went wrong.

Case 1: `fopen()` returns a NULL pointer on failure.

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    printf("Error opening file");
    exit(1);
}
```

Case 2: `fscanf()` returns the number of items successfully read. Check if the expected number of items was read.

```
int num;
if (fscanf(fp, "%d", &num) != 1) {
    fprintf(stderr, "Error reading integer from file\n");
}
```

Case 3: closing File safety

```
if (fclose(fp) != 0) {
    perror("Error closing file");
}
```

PS: Think of return code checking like **checking a receipt after shopping** — it helps you know if everything went right or if there was a billing error.

2. **Global Variable errno:** C programming being a system programming language, it provides you access at lower level in the form of return values from the functions. Most of the C function calls return -1 or NULL in case of any error and set an error code **global variable errno**. When a function is called in C, a variable named as `errno` is automatically assigned a code (value) which can be used to identify the type of error that has been encountered. Its a global variable **indicating the error occurred during any function call** and defined in the header file **errno.h**. Different codes (values) for `errno` mean different types of errors.

errno value	Type of Error
0	No Error
1	Operation not permitted
2	No such file or directory
5	I/O error
7	Argument list too long
9	Bad file number
11	Try again
12	Out of memory
13	Permission denied

The `errno` value indicate the types of error encountered. If you want to show the **error description**, there are **two functions that can be used to display a text message that is associated with `errno` - `perror()` and `strerror()`**

3. **`perror()` function:** The function `perror()` **stands for print error**. In case of an error, the programmer can determine the type of error that has occurred using the `perror()` function. It displays **the string you pass to it, followed by a colon, a space, and then the textual representation of the current `errno` value**.

Syntax: `void perror (const char *str) // (str: is a string containing a custom message to be printed before the error message itself.)`

Coding Example_1: Demo of perror()

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main (){
    FILE *fp = fopen ("data.txt", "r"); // File with this name exist
    //FILE *fp = fopen ("data1.txt", "r"); // File with this name doesn't exist
    printf("Value of errno: %d\n", errno);
    perror("Bad code");
    return 0;
}
```

```
C:\Users\Dell>a
Value of errno: 0
Bad code: No error
```

```
C:\Users\Dell>a
Value of errno: 2
Bad code: No such file or directory
```

4. **strerror() function:** The function used to get a **human-readable error message**. Returns a pointer to the textual representation of the current error code usually stored in the global variable `errno`. It is important to note that the **returned string is statically allocated** and should not be modified.

Syntax: `char *strerror(int errnum);`

Coding Example_2: Demo of strerror()

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
int main() {
    FILE *fp = fopen("data1.txt", "r"); // Trying to open a file that may not exist
    if (fp == NULL) {
        printf("File open failed: %s\n", strerror(errno));
    }
    return 0;
}
```

```
C:\Users\Dell>a
File open failed: No such file or directory
```

Coding Example_3: Usage of perror() and strerror() while handling illegal operations on the file

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main ()
{
    FILE *fp;
    fp = fopen ("data.txt", "r"); // File with this name exist
```

```
C:\Users\Dell>a
value of errno is 9 with info:Bad file descriptor
Bad code: Bad file descriptor
```



```
fputc('A',fp); // writing to a file which is opened for read
printf("value of errno is %d with info:%s\n",errno,strerror(errno));
perror("Bad code");
fclose(fp);
return 0;
}
```

To ensure safe and error-free file operations in C, two standard library functions are provided to monitor the status of a file stream. These functions help detect specific conditions such as reaching the end of a file or encountering an error during input/output operations. The two commonly used status-checking functions are:

1. **fEOF()** – This function checks if the end of the file (EOF) has been reached. It is typically used in loops to prevent reading past the file's end.
2. **ferror()** – This function checks whether an error has occurred during a file operation, such as reading from or writing to the file. It helps identify issues like invalid file access or hardware failures.

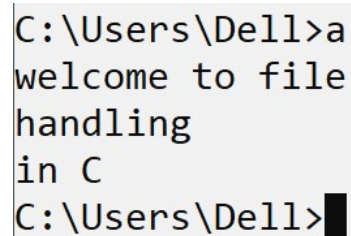
fEOF():

In C, `getc()` returns EOF when end of file is reached. `getc()` also returns EOF when it fails. So, only comparing the value returned by `getc()` with EOF is not sufficient to check for actual end of file. To solve this problem, C provides `fEOF()` function used to check whether the end of a file has been reached during a file read operation. This **returns a non-zero value only if end of file has reached, otherwise it returns 0.**

Syntax: `int fEOF(FILE *stream);`

Coding Example_4: Usage of fEOF() in loops to detect when a read operation failed.

```
#include <stdio.h>
int main ()
{
    FILE *fp = fopen("data.txt", "r");
    char ch;
    if (fp != NULL) {
        while (!fEOF(fp)) {
            ch = fgetc(fp);
            if (!fEOF(fp)) // check again after reading
                putchar(ch);
        }
    }
}
```



Content of file has been read and displayed.

```
    }  
}  
fclose(fp);  
return 0;  
}
```

feof() does not predict or detect EOF in advance. Instead, it checks if the last file operation **attempted to read beyond the end of the file. Best Practice is** instead of checking feof() before reading, check the return value of the read function itself (like fgetc() or fscanf()), and use feof() only to confirm EOF after a failed read.

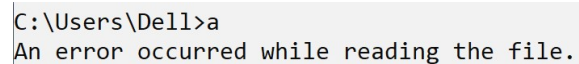
ferror():

This function checks for any error in the stream occurred during input or output operations. **It returns a value zero if no error has occurred and a non-zero value if there is an error.** The error indication will last until the file is closed unless it is cleared by the clearerr() function.

Syntax: int ferror (FILE *file_pointer);

Coding Example_5: Usage of ferror() to find the illegal operation on the file stream.

```
#include <stdio.h>  
int main() {  
    FILE *fp = fopen("data1.txt", "w"); // file is opened in write mode  
    char buffer[100];  
    if (fp != NULL) { // trying to read - illegal operation on the stream  
        if (fgetc(buffer, sizeof(buffer), fp) != NULL) {  
            printf("Read line: %s", buffer);  
        } else {  
            if (ferror(fp)) {  
                printf("An error occurred while reading the file.\n");  
            }  
        }  
    }  
    fclose(fp);  
}  
return 0;  
}
```



The `ferror()` is useful when a read or write operation fails. Instead of assuming the cause is end-of-file (EOF), this function helps determine whether the failure was due to an actual error such as:

- Reading from a corrupted file
- Writing to a full disk
- Accessing a file in an invalid mode

Best practice is that **the `ferror()` should only be used after performing input or output operations** such as `fgets()`, `fscanf()`, `fread()`, or `fwrite()`. It checks whether an error occurred during those operations and returns a non-zero value if an error has indeed happened. **It does not initiate the check by itself—it must follow a failed I/O attempt.** Additionally, because `ferror()` only signals that an error occurred without giving details, it's **recommended to use it in combination with `perror()` or `strerror(errno)` to retrieve a human-readable error message.** This is particularly helpful for debugging or logging errors for user-friendly messages. To distinguish between **an end-of-file condition and a real error**, it's also good practice to use `feof()` alongside `ferror()`.

Coding Example_6: Displaying the contents of the file on the terminal.

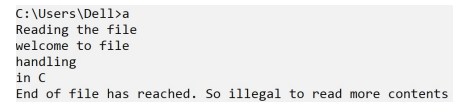
Version 1: Reading after encountering EOF

```
#include<stdio.h>
int main( )
{
    FILE *fp; char c;
    fp = fopen ( "data.txt", "r" ); // opening an existing file
    if ( fp != NULL )
    {
        printf( "Reading the file\n" );
        while ( !feof(fp) ) // returns nonzero if EOF encountered
        {
            c = fgetc(fp) ; // reading the file
            printf ("%c",c) ;
        }
        c = getc(fp); // doesn't show any error. so better to use feof function. See Version 2
        perror("\nBAD Code");
        fclose ( fp ) ; // Closing the file
    }
    return 0;
}
```

```
C:\Users\Dell>a
Reading the file
welcome to file
handling
in C
BAD Code: No error
```

Version 2:

```
printf( "Reading the file\n" );
while ( !feof(fp) ) // returns nonzero if EOF encountered
{
    c = fgetc(fp) ; // reading the file
    printf ("%c",c) ;
}
if(feof(fp))
{
    c = getc(fp);
    printf("\nEnd of file has reached. So illegal to read more contents\n");
}
fclose ( fp ) ; // Closing the file
```



```
C:\Users\Dell>a
Reading the file
welcome to file
handling
in C
End of file has reached. So illegal to read more contents
```

clearerr():

It is used to reset **the error and EOF indicators for a file stream**. After an error or end-of-file condition occurs on a stream, further operations might not proceed as expected until these indicators are cleared.

Coding Example_7: Writing to a file which is opened in read mode

```
#include <stdio.h>
#include <errno.h>
int main(){
    FILE* fp;
    char feedback[100];
    fp = fopen("data.txt", "r"); // opened in r mode. File must be existing
    printf("Provide feedback on the article: ");
    fgets(feedback, 100, stdin);
    for (int i = 0; i < feedback[i]; i++)
        fputc(feedback[i], fp); // writing to a file character by character
    printf("%d",errno);
    perror("Bad code"); //writing to a file is not allowed as the mode used is r while opening
    if (ferror(fp))
        printf("\nError writing to a file");
```

clearerr(fp); // error indication is stopped . next ferror() is returning 0

```
printf("\nAfter clearing fp\n");
if (ferror(fp)) {
    printf("Error again to write to a file\n");
}
return 0;
}
```

```
C:\Users\Dell>a
Provide feedback on the article: this article is good
9 Bad code: Bad file descriptor

Error writing to a file
After clearing fp
```

Think about this - > If we comment clearerr(fp), what happens?

```
C:\Users\Dell>a
Provide feedback on the article: good
9 Bad code: Bad file descriptor

Error writing to a file
After clearing fp
Error again to write to a file
```

The best practice is to use clearerr() **only after you have properly handled the error or end-of-file (EOF) condition**. This ensures that you're not ignoring unresolved issues that may affect the correctness of your program. This allows the file stream to be safely reused for further read or write operations without being blocked by previously set indicators.

Best practices to be followed during File handling

- Always Check File Pointers
- Validate Return Values of I/O Functions
- Use ferror() and feof() appropriately
- Clear Errors if needed
- Always Close Files
- Provide User-Friendly Error Messages
- Avoid Hard coding File Paths

Happy Error-Free File Handling implémentations!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #3
Problem solving using File handling functions***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Problem solving using File handling Functions**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Solve the below programs using File handling functions.

Link for Solution:

https://drive.google.com/file/d/1zak9bA_bak_wuu40iDBN_a2c5TVfUL1g/view?usp=drive_link

Data files: <https://drive.google.com/drive/folders/1nBeirvXZcvWr7N7gk0-CPWq7LuQ5ffbE?usp=sharing>

Level-1: Banana

1. Aryan wants to leave a short message for his friend in a file, so that his friend can read it later. Write a C program that **asks the user to enter a message (one line) and saves this message to a file named "message.txt"**. Display the content of message.txt on the terminal.
2. Ria has a file filled with mixed content—**letters, symbols, spaces, and numbers**. She wants to access the file content and **display only the digits (0–9)** from the file, ignoring everything else. Help Ria by writing a program that reads the file and prints all the digits present in it.
3. Arjun wants to write a C program that **reads a text file line by line, removes the newline character, appends a colon(:) at the end of each line, and displays the updated lines on the screen**. Help him implement this using file handling.

Input file:

Arjun likes coding
He practices every day
C is his favorite language

Expected output:

Updated lines:

Arjun likes coding:He practices every day:C is his favorite language:

4. Naina is a librarian at a small town library. Every evening, she updates a file with the list of books issued that day. She writes a C program to do the following:
 - **Accept the book title and the borrower's name from the user.**
 - **Write this information to a file named "issued_books.txt" in the format: Book: <title>, Borrower: <name>**
 - **Repeat this until the user types "stop" as the book title.**
5. Anvi maintains a small grocery shop. Every day, she enters the **details of items sold into a text file named sales.txt**. Each line in the file contains **Item name (string), Quantity sold (integer) and Price per item (float)**. Calculate the total sales amount for the day using a C language.

Level-2: Orange

6. Priya likes writing her thoughts in a digital diary every evening. At the end of each month, she wants to know how much she has written — **the number of characters, words, and lines** — **so she can track her writing habits.**

You are a computer science student helping Priya to read her diary file named diary.txt and print the total number of characters, total number of words and total number of lines.

7. Rahul is a huge fan of the Indian Premier League (IPL) and wants to analyze match statistics from past seasons. He has a **CSV file named matches.csv that contains data about all IPL matches.** Each row in the file contains details like **match ID, season, city and many more details.** Rahul is particularly interested in the inaugural season (2008). Help him write a C program to read the file and **count how many matches were played in the year 2008.**

You can assume that the file matches.csv is properly formatted. The second column in each row contains the match date in the format YYYY.

8. Anjali is a volunteer at her school library. She wants to build a small C program that **helps to record and save book information into a file. Each book has the following details - Title (string), Author (string) and Year of Publication (integer).** Write a C program that:

- Defines a structure Book with the above fields.
- Allows the user to enter the details of **N books.**
- Writes all the book details into a file named books.txt.
- Then, read the file and display the book details on the screen.

Level-3: Jackfruit

9. In the heart of TechSpark Robotics Lab, Thakshini, a brilliant computer science student was always finding clever ways to solve real-world problems. Her **project involved training humanoid robots to handle college administrative tasks like managing entry at tech fests and workshops.** These robots were deployed temporarily, but every evening, the lab manager, Mr. Rajan, found himself stuck - “I just can’t keep track of which robot worked today,” he sighed, flipping through a dog-eared notebook filled with scratched-out names and IDs. He asked “Thakshini, can your smart bots log themselves in next time?” That joke sparked Thakshini’s next mini project. She decided to make a text-based terminal program in C that:

- **Took the robot’s ID and name from the user.**
- **Stored them in a plain text file called employee.txt.**
- **Allowed Mr. Rajan to view all logged robots with just a few clicks.**

She wants to create a menu-based system:

- 1. Add Robot Staff**
- 2. View Robot Staff**
- 3. Exit**

Sample execution:

===== ROBOT STAFF CONTROL PANEL =====

1. Add Robot Staff
2. View Robot Staff
3. Exit

Enter your choice: 1

Enter the number of robot staff to log: 2

Enter Robot ID: 1001

Enter Robot Name: Xena

Enter Robot ID: 1002

Enter Robot Name: Bolt

Robot records added successfully!

===== ROBOT STAFF CONTROL PANEL =====

1. Add Robot Staff
2. View Robot Staff
3. Exit

Enter your choice: 2

--- LOGGED ROBOT STAFF DETAILS ---

Robot ID: 1001

Name: Xena

Robot ID: 1002

Name: Bolt

10. Meena, a lab assistant in the computer science department, is developing a simple login system for the lab's admin panel. **Only authorized users are allowed to change system configurations or access student records.** To begin with, she hard-codes the registered username as "student". Whenever a user tries to log in, the system should verify if the typed username matches. **To enhance the system further, Meena decides to** store all login attempts (both successful and failed) **in a file named login_log.txt**. Every time a user attempts to log in, the program should:

- Prompt the user to enter a username.
- Compare the entered username with the valid one ("student").
- Display a message: "Login successful" or "Access denied".
- **Append the attempt** and its result (Success or Failure) to login_log.txt along with the entered username.

Help Meena write a C program that performs this task using **file handling**.

A test of brilliance for exceptional students

ABC is a growing retail store that wants to digitize its product inventory and customer billing using simple C programs that work with .csv files. The system must track products, generate bills, maintain stock levels, and create sales reports.

Product data is stored in a file named products.csv:

Format -> ProductID (unique), ProductName, Category, Price, Stock

Sale data is stored in a file named sales.csv:

Format -> BillNo (unique to each purchase), ProductID (same as in products.csv), QuantitySold, TotalAmount

Banana:

1. Take product details (name, category, price, stock) from the user and add them to products.csv. Assume that the ProductID of a new product will be one more than the previous ProductID (100, 101, 102...).
2. Read all entries from products.csv and display them in a well-formatted table.
3. Take a product name as input and search for that product in products.csv. Display its details if found, otherwise display an appropriate message.
4. Use sales.csv to find the total amount made in sales so far, as well as the average bill value for each sale.
5. List every product category and the number of products in each.

Orange:

1. Record a purchase by taking product IDs and quantities for each product bought as input, and update the sales.csv and products.csv files accordingly. Include edge cases such as when the quantity trying to be bought exceeds the current stock of the product or when the stock of a product reaches zero after a purchase. Note: Multiple products being bought at once will all have the same bill number.
2. Sort all products by price and write the sorted list to a new file called sorted_products.csv.
3. Delete a product from products.csv using its product ID.
4. Add a stock alert feature that prints an alert message directly to the console when the stock of any given product goes below 10 units.

Jackfruit:

1. Identify and print the top 3 products that bring in the most revenue for company ABC, along with how much revenue they generated.
2. Implement a rewind function that takes n (number of sales) as a parameter, and undoes the effects of the last n transactions on both products.csv and sales.csv.

Data lives on, beyond your code — with files !!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #4
Searching on the file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Searching on the file data using array of pointers**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

When working with files in C, efficient data handling is crucial, especially when the file contains multiple entries such as lines, records, or words. One powerful technique is using an **array of pointers** to store and access the data dynamically. This approach **allows flexible memory management, faster search operations, and easier data manipulation without altering the original file content**. By storing each line or record in a dynamically allocated memory block and keeping track of these blocks through an array of pointers, we can perform searches or any modifications in-memory, leading to optimized and structured file processing.

Let us consider a simple text file called numbers.txt which contains many integers in an ascending order, separated by any white space in between. The file is well structured and very well formatted. The requirement is to search for the given integer in the file and display appropriate message.

Coding Example_1: Recursive binary search on an array having the integers from the file.

```
#include <stdio.h>
#include <stdlib.h>
int Search(int **aop, int low, int high, int key);
int main(){
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
        fclose(fp);
```

```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nEnter the integer to be searched\n");
int key_int;
scanf("%d", &key_int);
int res = Search(aop,0, n, key_int);
if(res == -1)
    printf("The key integer doesn't exist in the file\n");
else
    printf("Found at position %d", res);
}
return 0;
}
int Search(int **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else{
        int mid = (low+high)/2;
        if(*aop[mid] == key)
            return mid;
        else if(key < *aop[mid])
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

```

```

Displaying the contents of the dynamic array using array of integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Displaying the contents of the dynamic array using array of pointers to integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Enter the integer to be searched
89
The key integer doesn't exist in the file
C:\Users\Dell>a

Displaying the contents of the dynamic array using array of integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Displaying the contents of the dynamic array using array of pointers to integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Enter the integer to be searched
30
Found at position 2

```

Now let us consider the array of structures and array of pointers to array of structures. Given data file - > **student_rollno_name.csv**. If the roll_number of the student is given, display the details of that student.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

Coding Example_2: Searching the details of student based on roll_number

```
typedef struct student
{
    int roll;      char name[20];  }STUDENT;
void init_ptr(struct student s[], struct student *p[], int n);
void disp(struct student* p[], int n) ;
int Search(STUDENT **aop, int low, int high, int key);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    FILE *fr=fopen("sorted_data.csv","r");
    if(fr == NULL) {
        printf("Cant access the file\n");exit(1);
    }
    char a[200];  fgets(a,200,fr);      char *item; int i=0;
    struct student s[100];              // Array of structures
    // Reading a line from the file and splitting it based on the delimiter comma(,), copying the details
    to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,",");      s[i].roll=atoi(item);
        item=strtok(NULL,",");   strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n); // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n);      // to display the array of structures using array of pointers
    int key_rollno;
    printf("Enter the roll number of the student to be searched\n");
    scanf("%d",&key_rollno);
    int res = Search(p, 0, n-1, key_rollno);
    if(res == -1)
        printf("Student not found\n");
    else{
        printf("Student details are here\n");
        printf("Roll_number %d, Name is %s",p[res]->roll, p[res]->name);
    }
    return 0;
}
```



```
int Search(STUDENT **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else {
        int mid = (low+high)/2;
        if(aop[mid]->roll == key)
            return mid;
        else if(key < aop[mid]->roll)
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0 ; i < n; i++)
        p[i] = &s[i];
}

void disp(struct student* p[], int n) {
    for(int i = 0 ; i < n; i++)
        printf("%d %s", p[i]->roll, p[i]->name);
}
```

```
Enter the roll number of the student to be searched
105
Student not found
```

```
Enter the roll number of the student to be searched
43
Student details are here
```

Think about this-> How to compare the strings if user wants to enter the student name and get other details of the student?

Happy Searching file contents using Array of Pointers!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #5
Sorting file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Sorting file data using array of pointers**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

When working with text files, sorting the content such as integers or words or line of strings can be efficiently managed using an **array of pointers**. Instead of moving entire content during sorting, we manipulate the pointers to those contents, which **reduces memory operations and improves performance**. This approach involves reading file contents into dynamically allocated memory blocks, storing addresses of these blocks in an array of pointers, and then applying a sorting algorithm (like selection sort) on the pointer array. This technique is especially useful for handling large text files with multiple lines or records, offering **both speed and flexibility in data processing**.

Let us consider a simple text file called numbers.txt which contains a lot of integers in a well structured manner. The requirement is to sort the integers in the file and write the sorted collection of integers to another file.

Coding Example_1:

```
#include <stdio.h>
#include <stdlib.h>
void swap( int** lhs, int** rhs );
void sort(int **aop, int n);
int main(){
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
```

```
Displaying the contents of the dynamic array using array of integers
100    500    200    6    60    70    80    90
Displaying the contents of the dynamic array using array of pointers
s
100    500    200    6    60    70    80    90
After sorting
Displaying the contents of the dynamic array using array of integers
100    500    200    6    60    70    80    90
Displaying the contents of the dynamic array using array of pointers
s
6    60    70    80    90    100    200    500
Writing to a new file
Writing to a new file is done.
```

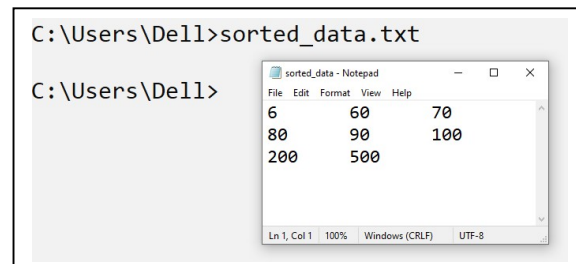
```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
sort(aop, n);
printf("\nAfter sorting\n");
printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nWriting to a new file\n");
FILE *fpw = fopen("sorted_data.txt", "w");
for(int i = 0 ; i < n; i++)
    fprintf(fpw, "%d\t", *aop[i]);
printf("\nWriting to a new file is done.\n");
fclose(fp);
fclose(fpw);
}
return 0;
}

void sort(int **aop, int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(*aop[pos] > *aop[j])        pos = j;
        }
        if(pos != i)
            swap(&aop[pos],&aop[i]); // Swap the pointers not the integers in the array
    }
}

void swap( int** lhs, int** rhs){
    int* temp = *lhs;    *lhs = *rhs;    *rhs = temp;
}

```



Now let us consider the array of structures and array of pointers to array of structures. Given a data file **student_rollno_name.csv**, create a menu driven program to perform selection sort based on roll_number and name. Write the sorted record to a new file.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

Coding Example_2:

```
struct student
{
    int roll;
    char name[20];
};

void init_ptr(struct student s[], struct student *p[], int n);
void swap( struct student** lhs, struct student** rhs);
void disp(struct student* p[], int n) ;
void selection_sort_roll_no(struct student* s[],int n);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    FILE *fr=fopen("student_rollno_name.csv","r");
    char a[200]; fgets(a,200,fr);    char *item;
    struct student s[100]; int i=0;
    // Reading a line from the file and splitting it based on the delimiter comma(,),
    copying the details to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,"");
        s[i].roll=atoi(item);
        item=strtok(NULL,"");
        strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n); // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n); // to display the array of structures using array of pointers
```

// Menu driven code

```
int ch;
printf("enter the choice.\n 1.sort on roll\n");
printf("2. sort on name");
scanf("%d",&ch);
switch(ch) {
    case 1:selection_sort_roll_no(p,n); // array of pointer and n is the argument
           disp(p,n); break;
    case 2:selection_sort_names(p,n); disp(p,n); break; // same as above
    default: printf("exiting from the program"); exit(0);
}
```

// Writing the sorted record to a new file

```
FILE *fw=fopen("student_sorted.csv","w");
fprintf(fw,"Roll_number,Name\n");
i=0;
while(i<n){
    fprintf(fw,"%d,%s",p[i]->roll,p[i]->name); i++;
}
fclose(fw);
printf("sorted data is written to a file student_sorted.csv\n");
return 0;
}
```

// Initializing the pointer

```
void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0; i < n; ++i)
        p[i] = &s[i];
}
```

// Swapping the pointers

```
void swap( struct student** lhs, struct student** rhs) {
    struct student* temp = *lhs; *lhs = *rhs; *rhs = temp;
}
```

//Displaying using array of pointers

```
void disp(struct student* p[], int n) {
    for(int i = 0; i < n; ++i)
        printf("%d %s\n", p[i]->roll, p[i]->name);
    //printf("%s %d\n", (s+i)->name, (s+i)->roll);
}
```

//Selection sort using roll_no on array of pointers to array of structures

```
void selection_sort_roll_no(struct student *s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        printf("in for\n");
        pos = i;
        for(j = i+1;j<n;j++)    {
            if((s[pos]->roll) > (s[j]->roll)) // compare the roll numbers using pointers to
sort the record
                pos = j;
        }
        if(pos != i)
            swap(&s[pos],&s[i]);    // Swap the pointers not the structures
    }
}
```

//Selection sort using name on array of pointers to array of structures

```
void selection_sort_names(struct student* s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(strcmp(s[pos]->name,s[j]->name)>0)
                pos = j; // compare the names using pointers to sort the record
        }
        if(pos != i)
            swap(&s[pos],&s[i]);
    }
}
```

Happy Sorting file contents using Array of Pointers!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #6
Searching and Sorting File data using callback***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Searching and Sorting File data using callback**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

In continuation with our earlier discussion on searching and sorting, if searching and sorting needs to be done based on two different criteria, **a callback function can be used to define the searching and sorting logic dynamically**. This approach avoids the need to write separate sorting functions for each criterion.

Coding Example_1: Recursive binary search on an array having the integers from the file using callback and array of pointers.

```
#include<stdio.h>
#include<stdlib.h>

// added in this version
int is_even(int x) // first criteria for searching, number must be even
{
    return x%2 == 0; }
int is_less_than_22(int x) // second criteria – number must be less than 22
{
    return x<22; }

int my_search(int a[],int low,int high,int key,int(*p)(int))
{
    // recursive solution
    int pos = -1; int mid;
    if (low>high)
        return pos;
    else
    {
        mid = (low+high)/2;
        if(a[mid]==key && p(key))
            pos = mid;
        else if(a[mid]>key)
            return my_search(a,low,mid-1,key,p);
        else
            return my_search(a,mid+1,high,key,p);
    }
    return pos;
}

int main()
{
    FILE *fp = fopen("numbers.txt", "r");
```

```
C:\Users\Dell>a
Enter the key to be searched in the file content
100
It is even and found at 9 position
not found

C:\Users\Dell>a
Enter the key to be searched in the file content
1000
not found
not found
```

```
if(fp == NULL)
    printf("Cant access the file\n");
else
{
    int a;          int n = 0;
    while(fscanf(fp, "%d", &a) != -1)
        n++;
    int *p = malloc(n*sizeof(int));
    int **aop = (int **)malloc(n * sizeof(int *));
    for(int i = 0; i < n; i++)
        aop[i] = &p[i];
    rewind(fp);
    int i = 0;
    while(fscanf(fp, "%d", &p[i]) != -1)
        i++;
    fclose(fp);

    int key;
    printf("Enter the key to be searched in the file content\n");
    scanf("%d",&key);
    // perform search on a collection of elements and print only if it is even
    int pos = my_search(p,0,n-1,key,is_even);
    //printf("result is %d",pos);
    if(pos !=-1)
        printf("It is even and found at %d position\n",pos);
    else
        printf("not found\n");

    // perform search on a collection of elements and print only if it is less than 22
    pos = my_search(p,0,n-1,key,is_less_than_22);
    //printf("result is %d",pos);
    if(pos !=-1)
        printf("It is less than 22 and found at %d position\n",pos);
    else
        printf("not found\n");
    fclose(fp);
}

return 0;
}
```

Coding Example_2: Menu based code to apply Selection sort on an array of pointers using callback and writing the sorted data to a new file. Open the sorted file and observe the contents

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
typedef struct Student
{
    int roll;    char name[10];    }ST;
void init_ptr(ST *s,ST **sp,int n);
void display(ST *s, int n);
void display_ptr(ST **sp, int n);
void swap(ST **s1, ST **s2);
void sort(ST **sp,int n, int (*comp)(ST *, ST *));
void write_to_file(FILE *fpw, ST **sp,int n);

//New functions added now
int comp_roll(ST *s1, ST *s2)
{
    return s1->roll > s2->roll;    }
int comp_name(ST *s1, ST *s2)
{
    return strcmp(s1->name,s2->name)>0;    }

int main()
{
    FILE *fp = fopen("student_rollno_name.csv", "r");
    FILE *fpw = fopen("student_rollno_name_new.csv", "w");
    if(fp == NULL || fpw == NULL)
        printf("Cant access the file");

    else{
        int n = 0; char line[2000];
        while(fgets(line,2000, fp) != NULL)
            n++;
        //printf("Number of rows %d", n);
        rewind(fp);
        ST *s = malloc(n*sizeof(ST));
        fgets(line,2000, fp);
        int i = 0;
        while(fgets(line,2000, fp) != NULL)
        {
```

```
C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
1
Successfully written the sorted data to a file

C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
2
Successfully written the sorted data to a file

C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
4
invalid input!!
```

```
s[i].roll = atoi(strtok(line,","));
strcpy(s[i].name, strtok(NULL,","));
i++;
}
ST **sp = malloc(n*sizeof(ST*));
init_ptr(s,sp,n);
//display(s,n-1);
//display_ptr(sp,n-1);
printf("press 1 to sort using roll_no\n");
printf("press 2 to sort using name\n");
int choice;
scanf("%d", &choice);
switch(choice)
{
    case 1:sort(sp,n-1, comp_roll); //display_ptr(sp,n-1);
           break;
    case 2:sort(sp,n-1, comp_name); //display_ptr(sp,n-1);
           break;
    default:printf("invalid input!!");exit(0);
}
write_to_file(fpw, sp,n-1);
fclose(fpw);
}
}

void sort(ST **sp,int n, int (*comp)(ST *, ST *)) // Function pointer in the parameter
{
    int i,j,pos;
    for(i = 0; i < n-1; i++)
    {
        pos = i;

        for(j = i+1; j<n;j++)
        {
            if(comp(sp[pos],sp[j]))
                pos = j;
        }
        if(pos != i)
            swap(&sp[pos], &sp[i]);
    }
}
```

```
void swap(ST **s1, ST **s2){
    ST *temp = *s1;    *s1 = *s2;    *s2 = temp;
}

void init_ptr(ST *s, ST **sp, int n){
    for(int i = 0 ; i < n;i++)
    {
        sp[i] = &s[i];  }
}

void display(ST *s, int n){
    for(int i = 0; i < n; i++)
        printf("%d %s\n", s[i].roll, s[i].name);
}

void display_ptr(ST **sp, int n){
    for(int i = 0; i < n; i++)
        printf("%d %s\n", sp[i]->roll, sp[i]->name);
}

void write_to_file(FILE *fpw, ST **sp,int n){
    for(int i = 0; i < n; i++)
        fprintf(fpw, "%d,%s", sp[i]->roll, sp[i]->name);
    printf("Successfully written the sorted data to a file\n");
}
```

One callback, endless Searching and Sorting possibilities!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #7
Creation and Usage of Header files***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Creation and Usage of Header files**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

In C programming, header files serve as a way to organize code for better **modularity, readability, and reusability**. A header file typically contains **function declarations, macro definitions, constants, and structure or type definitions that are used across multiple source files**. C provides several **predefined header files** like `<stdio.h>`, `<math.h>`, and also allows you to create **custom header files** to include user-defined functions and types. **Header files are included in the program using the `#include` directive**.

Advantages of using header files:

- Avoiding rewriting the code across different .c files.
- Separating declarations(interface) from definitions(implementation), keeping code clean and organized.
- Easier maintenance and collaboration in large projects.

Let us consider an example code related to **finding whether the number is a palindrome or not**.

Coding Example_1: All operations are in one single file.

```
#include<stdio.h>
int main()
{
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    int rev = 0;    int number = n;
    while(n>0) // can be just while(n).
    {
        rev = rev * 10 + n % 10; n /= 10;
    }
    if(number == rev)
        printf("%d is palindrome\n",number);
    else
        printf("%d is not palindrome\n",number);
    return 0;
}
```

```
C:\Users\Dell>a
Enter the number
-111
-111 is not palindrome

C:\Users\Dell>a
Enter the number
12121
12121 is palindrome

C:\Users\Dell>a
Enter the number
1
1 is palindrome
```

Coding Example_2: Separating the interface and implementation - Function to check whether the given number is palindrome or not.

```
include<stdio.h>
int is_palin(int); // interface
int main(){
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

```
C:\Users\Dell>a
Enter the number
123
123 is not a palindrome

C:\Users\Dell>a
Enter the number
-3443
-3443 is not a palindrome

C:\Users\Dell>a
Enter the number
15351
15351 is a palindrome
```

Now let us create separate files for interface and implementation to follow modular programming practices and enable code reuse and maintainability.

Coding Example_3: Creation of header file. Separating the client and server.

//palin.h

int is_palin(int); // interface

//client code – palin_client.c

#include<stdio.h>

```
int main(){
    int n;
    printf("Enter the number\n"); scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
```

```
C:\Users\Dell>gcc -c palin_client.c
C:\Users\Dell>gcc -c palin.c
C:\Users\Dell>gcc palin_client.o palin.o
C:\Users\Dell>a.exe
Enter the number
121
121 is a palindrome

C:\Users\Dell>a.exe
Enter the number
-121
-121 is not a palindrome

C:\Users\Dell>a.exe
Enter the number
5
5 is a palindrome
```

//Server code Implementation – palin.c

```
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

Execution Steps when using Header and Implementation Files in C.

1. Compile Each .c File into an Object File (.o)
2. Link All .o Files to Create the Final Executable
3. Run the Executable

For the above codes,

gcc -c palin.c palin_client.c // results in 2 .o files

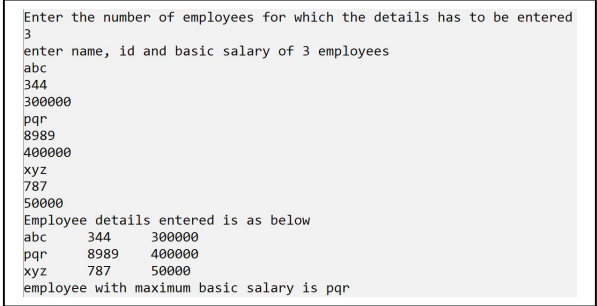
gcc palin.o palin_client.o // results in an executable file

a.exe OR ./aout // Execute the executable

Let us consider an application in C to read and display the details of n Employees. Separate the interface and implementation.

Coding Example_4: Employee application segregating client and server and display the employee details whose salary is max.

```
//employee.h
typedef struct employee
{
    char name[20];
    int id;
    int basic_salary;
    int total_salary;
}employee_t;
void scan_details(employee_t*, int);
void display_details(const employee_t*, int);
int calculate_total_salary(employee_t*, int,int);
employee_t max(employee_t*, int);
```



```
Enter the number of employees for which the details has to be entered
3
Enter name, id and basic salary of 3 employees
abc
344
300000
pqr
8989
400000
xyz
787
50000
Employee details entered is as below
abc      344      300000
pqr      8989      400000
xyz      787      50000
employee with maximum basic salary is pqr
```

```
// employee_server.c

#include "employee.h"
#include <stdio.h>

void scan_details(employee_t* e, int n){
    for(int i = 0; i < n; i++){
        scanf("%s %d %d", (e+i)->name, &(e+i)->id, &(e+i)->basic_salary);
    }

    void display_details(const employee_t* e, int n){
        for(int i = 0; i < n; i++){
            printf("%s\t%d\t%d\n", (e+i)->name, (e+i)->id, (e+i)->basic_salary);
        }
    }

    employee_t max(employee_t* e, int n){
        int i;    int max = 0;    employee_t e_max;
        for(i = 0; i < n; i++){
            if((e+i)->basic_salary > max) {
                max = (e+i)->basic_salary;        e_max = *(e+i);
            }
        }
        return e_max;
    }
}

//employee_client.c
#include <stdio.h>
#include "employee.h"
int main()
{
    employee_t e[1000]; int n, id;
    printf("Enter the number of employees for which the details has to be entered\n");
    scanf("%d", &n);
    printf("enter name, id and basic salary of %d employees\n", n);
    scan_details(e, n);
    printf("Employee details entered is as below\n");
    display_details(e, n);
    employee_t e_max = max(e, n);
    printf("employee with maximum basic salary is %s", e_max.name);
    return 0;
}
```

Keep your code neat — headers to the rescue!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #8
Environment Variables***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Environment Variables**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Environment variables are dynamic values that affect the way processes run on a computer. They are part of the environment in which a process executes and can influence the behavior of software and operating systems. They are typically used for **storing system-wide or user-specific settings**, pass configuration data to applications (like file paths, user details, or system preferences) and **control program behavior without hard coding** values in the program.

Examples of common environment variables include:

PATH – Specifies directories to search for executable files.

HOME or USERPROFILE – Indicates the user's home directory.

TEMP or TMP – Path for storing temporary files.

Widely used in operating systems like **Windows, Linux, and macOS**. Can be accessed or modified using shell commands or using C programs.

Standard Environment Variables

Environment variables store information such as the **user's home directory, terminal type, and current locale**. The collection of all environment variables with assigned values is referred to as the **environment**. These variable names are **case-sensitive and must not include the '=' character**. Their values can be any string, but must not contain NULL character.

The **number of environment variables in a system is not fixed** — it varies depending on **Operating system (Windows, Linux, macOS), User configuration (user-defined variables), System services, installed applications (they may add their own variables) and Shell session or login context**.

There are two ways in which we can display all environment variables.

Method – 1: Using an array of character pointers specified in main()

```
int main(int argc, char* argv[], char* envp[])
```

Method – 2: Using environ variable declared in stdlib.h.

char environ** is represented as an array of strings. Each string is of the format **'name=value'**.

Coding Example_1: Printing two environment variables

```
#include <stdio.h>
int main(int argc, char *argv[], char *envp[]){
    printf("%s\n", envp[0]);
    printf("%s\n", envp[1]);
    return 0;
}
```

```
C:\Users\De11\P2\Unit4>a
ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\De11\AppData\Roaming
```

Coding Example_2: Printing all environment variables using method-1

```
int main(int argc, char *argv[], char *envp[]){
    int i = 0;
    while(envp[i] != NULL){
        printf("%d %s\n", i, envp[i]); i++;
    }
    return 0;
}
```

Sample only. Many more there.

```
C:\Users\De11\P2\Unit4>a
0 ALLUSERSPROFILE=C:\ProgramData
1 APPDATA=C:\Users\De11\AppData\Roaming
2 CommonProgramFiles=C:\Program Files (x86)\Common Files
3 CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
4 CommonProgramW6432=C:\Program Files\Common Files
5 COMPUTERNAME=DESKTOP-S8KH1MP
6 ComSpec=C:\Windows\system32\cmd.exe
```

Coding Example_3: Printing all environment variables using method-2

```
#include<stdlib.h>
int main(){
    char **p = environ; int i = 0;
    while((*p) != NULL) {
        printf("%d %s\n", i, *p); i++;
        p++;
        //sleep(2);
    }
    return 0;
}
```

Sample only. Many more there.

```
C:\Users\De11\P2\Unit4>a
0 ALLUSERSPROFILE=C:\ProgramData
1 APPDATA=C:\Users\De11\AppData\Roaming
2 CommonProgramFiles=C:\Program Files (x86)\Common Files
3 CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
4 CommonProgramW6432=C:\Program Files\Common Files
5 COMPUTERNAME=DESKTOP-S8KH1MP
6 ComSpec=C:\Windows\system32\cmd.exe
```

Accessing and Modifying Environment Variables

The process of accessing and modifying environment variables is carried out using functions such as `getenv()`, `putenv()` and `setenv()`. Let us discuss these functions in detail.

getenv():

- Used to retrieve the value of a specific environment variable.
- **Prototype:** `char *getenv(const char *name);`
- Returns a pointer to the value associated with name, or NULL if it doesn't exist.

Coding Example_4:

```
int main(){
    char *path = getenv("PATH");
    if (path != NULL)
        printf("PATH: %s\n", path);
    else
        printf("PATH not found\n");
}
```

```
C:\Users\De11>a
PATH: C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;
```

setenv()

- Used to set or update an environment variable.
- **Prototype:** `int setenv(const char *name, const char *value, int overwrite);`
- If overwrite is non-zero, the variable is updated if it exists. If zero, it won't overwrite existing values.
- A null pointer for the value parameter is illegal.
- If the function is successful it returns 0. Otherwise the environment is unchanged and the return value is -1.

Coding Example_5: Appending the PATH with current directory(.)

```
include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    char *path = getenv("PATH");
    char path1[10000];
    strcpy(path1, path); strcat(path1, ".");
    printf("Before setting %s\n", getenv("PATH"));
    int i = setenv("PATH",path1, 1);
    printf("\n\nAfter setting %s\n", getenv("PATH"));
    printf("status of function is %d", i);
    return 0;
}
```

```
C:\Users\De11>a
Before setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;
After setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;.
status of function is 0
```

Points to think:

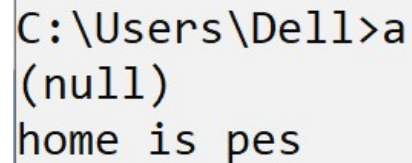
- If the last argument of setenv is 0, will the PATH be updated? - No
- What is the return value of setenv in this case? – 0(success)
- Can you create a new environment variable? If yes, what is the last argument?

putenv()

- Another way to modify the environment.
- **Prototype:** `int putenv(char *string);`
- The string must be in the format "NAME=VALUE" and must persist.

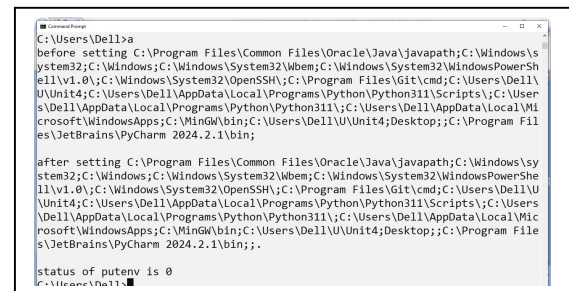
Coding Example_6: Adding a new environment using putenv

```
int main() {  
    printf("%s\n", getenv("HOME"));  
    putenv("HOME=pes");  
    char *p = getenv("HOME");  
    printf("home is %s", p);  
    return 0;  
}
```



Coding Example_7: Appending the new path to existing PATH

```
int main() {  
    char *path = getenv("PATH");  
    char mypath[10000];  
    strcpy(mypath, path); strcat(mypath, ";");  
    char path1[10000];  
    strcpy(path1, "PATH="); strcat(path1, mypath);  
    printf("before setting %s\n", getenv("PATH"));  
    int i = putenv(path1);  
    printf("\nafter setting %s\n", getenv("PATH"));  
    printf("\nstatus of putenv is %d", i);  
    return 0;  
}
```



Try this -> Display the list of all environment variables once after the new variable is added.

Modifying environment variables just got simpler !!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #9
Pre-Processor Directives***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Pre-processor Directives**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

The execution of a C program follows a multi-stage process that includes: Preprocessing, Compilation, Linking, Loading, and Execution. The first stage, preprocessing, is carried out by the C preprocessor. Preprocessor directives in C are instructions that are **executed before the actual compilation of the program begins**. Each directive starts with a **# symbol** and these directives are **commonly used to include header files, define constants and macros, and control conditional compilation**. You can observe the effect of the preprocessor on source files by using the **-E option with the gcc compiler**. Although it plays a crucial role in preparing the code, it operates independently from the compiler.

Key Types of Preprocessor Directives

- **Macro Definition**
- **File Inclusion**
- **Macro Undefinition**
- **Conditional Compilation**
- **Other Directives**

Macro Definition :

A macro is a fragment of code defined using the **#define** directive that gets replaced by its value or expression during the preprocessing stage. **#define** is used **to define constants or expressions that are replaced by their values before compilation**. Begins with the **#define** keyword and **does not require a semicolon** at the end.

Characteristics of a Macro:

1. **Textual Substitution/ Macros doesn't judge anything:** The preprocessor replaces all occurrences of the macro name with its defined value or expression before the actual compilation begins.
2. **Macros can have expression: No Data Type Enforcement:** Macros are **not type-checked**, unlike variables or functions. This can lead to unexpected behavior if not used carefully.

3. **No Memory Allocation:** Macros do not consume memory during program execution.
4. **Improves Code Readability:** Replacing magic numbers or repeated expressions with macros can make code easier to read and maintain.
5. **Parentheses are Crucial:** Always use parentheses around macro arguments and the entire macro body to avoid precedence errors during expansion.
6. **Can Take Arguments (Function-like Macros):**
7. **Cannot be debugged easily:** Since macros are replaced before compilation, they don't appear in the debug information, making them harder to trace.
8. **Can be Undefined:** Use `#undef` to remove a macro definition.

Advantages of Macros:

- **Faster Execution:** Macros are replaced directly in the code, so there's no function call overhead.
- **No Memory Usage for Storage:** Unlike variables, macros don't use memory at runtime since they are just text replacements.
- **Easier to Update Values:** If a constant value is used in many places, we can change it in one macro definition instead of everywhere.
- **Flexible and Reusable:** Function-like macros can be reused for common operations like squaring or finding the maximum.
- **Conditional Compilation:** Macros help write code that works on different systems or conditions by including or excluding parts of the code.

NOTE: Use `gcc -E filename.c` to view the output of preprocessing stage on the terminal.

Coding Example_1: Macros doesn't judge anything. Pure textual substitution

```
#include<stdio.h>
#define PI 3.14           // symbolic constant, macro, plain text substitution
// It doesn't judge anything
// No memory is allocated for this
int main()
{
    printf("%f",PI); return 0;
}
```

```
int main()
{
    printf("%f",3.14); return 0;
}
```

Coding Example_2: No memory allocated for macros. Addressing access is a Compile-time error

```
#define MAX 10
int main()
{
    int a[MAX];
    scanf("%d", &MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    int a[10];
    scanf("%d", &10);
    return 0;
}
```

Compilation Error:

```
check.c:5:13: error: lvalue required as unary '&' operand
    scanf("%d", &MAX);
                  ^
```

Coding Example_3: Defining a string using macro

```
#define NAME "Admin - Sindhu R Pai"
int main()
{
    printf("Welcome %s", NAME);
}
// Replace %s with %d and see the output.
```

Preprocessing stage output:

```
int main()
{
    printf("Welcome %s", "Admin - Sindhu R Pai");
}
```

Final output:

```
C:\Users\Dell>a
Welcome Admin - Sindhu R Pai
```

Coding Example_4: Macro having an expression

```
#include<stdio.h>
#define EXPR 2+5*1
int main()
{
    printf("Value is %d\n",EXPR);
    //STR replaced with 2+5*1 in
    //preprocessing stage.
    //During execution, expression is evaluated
    return 0;
}
```

Preprocessing stage output:

```
int main()
{
    printf("Value is %d\n",2+5*1);

    return 0;
}
```

Final output:

```
C:\Users\Dell>a
Value is 7
```


Coding Example_5: Macro with parameters

```
#include<stdio.h>
#define SUM(a,b) a+b // SUM is not a function. It is a parameterized macro.
int main()
{
    int p,q;    printf("Enter two numbers:\n");
    scanf("%d%d",&p,&q);
    printf("The sum of two numbers is %d\n",SUM(p,q));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    int p,q;
    printf("Enter two numbers:\n");
    scanf("%d%d",&p,&q);
    printf("The sum of two numbers is %d\n",p+q);
    return 0;
}
```

Coding Example_6: Usage of parentheses is crucial

```
#include<stdio.h>
#define sqr(x) (x*x) //change this to (x)*(x).
int main()
{
    printf("%d",sqr(2+3)); // 11(2+3*2+3)
                        // Not (2+3)*(2+3)
    return 0;
}
```

Pre-processing stage output:

```
int main()
{ printf("%d", (2+3*2+3));
  return 0;
}
```

Think -> What changes to be done in the above code to perform $(2+3)*(2+3)$?

Coding Example_7: Usage of parentheses is crucial

```
#include<stdio.h>
#define sqr(x) (x)*(x)
{
    printf("%d",sqr(2+3));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{ printf("%d", (2+3)*(2+3));
  return 0;
}
```

Coding Example_8: Usage of one macro in another

```
#define sqr(x) x*x
#define cube(x) sqr(x)*x // using sqr in cube
int main()
{
    printf("The cube of 9 is d\n",cube(9));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("The cube of 9 is d\n",9*9*9);
    return 0;
}
```

Coding Example_9: Redefining the macro using assignment operator results in Error

```
#define MAX 200
int main()
{
    printf("Before change %d\n", MAX)
    MAX=20;
    // Error because no memory allocated for MAX
    printf("After change %d",MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("Before change %d\n", 200)
    200=20;
    printf("After change %d",200);
    return 0;
}
```

Results in Compile time Error:

```
check.c: In function 'main':
check.c:5:5: error: lvalue required as left operand of assignment
    MAX=20;
    ^
```

Coding Example_10: Redefining a macro with #define is allowed. But not advisable.

```
#define MAX 200
int main()
{
    printf("Before change %d\n", MAX);
    #define MAX 20
    printf("After change %d",MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("Before change %d\n", 200);check.c:5:0: warning: "MAX" redefined
    #define MAX 20
check.c:1:0: note: this is the location of the previous definition
    #define MAX 200

    printf("After change %d",20);
    return 0;
}
```

Results in Warning at compiletime.

```
check.c:5:0: warning: "MAX" redefined
    #define MAX 20
check.c:1:0: note: this is the location of the previous definition
    #define MAX 200
```

Differences between a Macro and an Enumeration

Aspect	Macro (#define)	Enum (enum)
Definition	Preprocessor directive (text replacement)	Language-level data type
Type Safety	No type checking (just replaces text)	Type-safe (values have an integer type)
Scope	Global (no scope control)	Limited to scope where defined
Value Assignment	Must be manually assigned	Auto-incremented from 0 (or custom values) if value not spec
Readability & Maintenance	May reduce clarity when used excessively	More readable for related named constants

Similarity between macro and enum in C:

- **Both are used to define named constants:** Macros like `#define MAX 100` and enums like `enum { MAX = 100 };` assign meaningful names to fixed values, improving code readability.
- **The address operator (&) cannot be used on either:** Since macros are replaced by text and enums are not variables stored in memory, you cannot use the `&` operator to get their address.

Pre-defined Macros

Predefined macros in C are special identifiers that are automatically defined by the compiler. They provide useful information about the compilation environment such as **the current file name, line number, date, and time of preprocessing**. These macros begin with **double underscores (__)** and are mainly used for **debugging, logging, or conditional compilation**. You don't need to define them manually — they are available by default in every C program.

- ☐ **__FILE__**: Current file name
- ☐ **__LINE__**: Current line number
- ☐ **__DATE__**: Compilation date
- ☐ **__TIME__**: Compilation time
- ☐ **__STDC__**: Defined as 1 if the compiler follows ANSI C standard

Coding Example_11: Usage of Pre-defined macros

```
#include <stdio.h>
int main() {
    printf("File Name : %s\n", __FILE__);
    printf("Line Number : %d\n", __LINE__);
    printf("Date : %s\n", __DATE__);
    printf("Time : %s\n", __TIME__);
    printf("ANSI C? : %d\n", __STDC__);
    return 0;
}
```

Preprocessing stage output

```
:
# 3 "check.c"
int main() {
    printf("File Name : %s\n", "check.c");
    printf("Line Number : %d\n", 5);
    printf("Date : %s\n", "May 19 2025");
    printf("Time : %s\n", "15:50:46");
    printf("ANSI C? : %d\n", 1);
    return 0;
}
```

After executing the code:

```
C:\Users\Dell>a
File Name : check.c
Line Number : 5
Date : May 19 2025
Time : 15:46:08
ANSI C? : 1
```

Uses of Pre-defined Macros in C

- **Debugging:** Print the current file name, line number, or function to trace issues using `__FILE__`, `__LINE__`, and `__func__`.
- **Logging and Error Reporting:** Automatically log the date and time of preprocessing using `__DATE__` and `__TIME__`.
- **Conditional Compilation:** Use macros like `__STDC__` to check if the compiler follows the ANSI C standard and conditionally include code.
- **Code Documentation:** Embed build details in the output (like version/date/time) for tracking code versions or builds.
- **Cross-Platform Code Support:** Help write code that adapts to different environments or compiler standards using built-in conditions.

File Inclusion Directives: Used to include the contents of one file into another before compilation. This is done using the **#include** preprocessor directive.

- **Standard Library Files or Header files:** This contain the declarations of predefined functions like `printf()`, `scanf()`, etc. The corresponding header files must be included using the `#include` directive. The syntax `#include <file_name>` instructs the compiler to search for the file in the standard system directories (as defined by the compiler's include path).

Example: `#include <stdio.h> #include <string.h>`

- **User-defined files:** When a program becomes very large, it is a good practice to divide it into smaller files and include whenever needed. These types of files are user defined files. These files can be included as `#include "filename"`

Example: `#include "sort.h"`

Note: Conventionally, angle brackets `< >` are used for standard header files and double quotes `" "` for user-defined header files. This behavior can be altered by modifying the environment variables or include path settings, allowing user-defined files to be included using angle brackets as well.

Macro Undefinition:

Macros are defined using `#define`, but sometimes it becomes necessary to **remove or cancel** a macro definition. This is done using the `#undef` directive. It is useful when you want to **redefine a macro with a new value** or **avoid name conflicts** in large programs.

Coding Example_12: Undefining a macro

```
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    #undef MAX // Remove the existing macro definition  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Preprocessing stage output:

```
int main() {  
    printf("Before undefining, VALUE = %d\n", 10);  
  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Compilation Error:

```
check.c:6:47: error: 'MAX' undeclared (first use in this function)  
    printf("After undefining, VALUE = %d\n", MAX);  
                                              ^~~~~  
check.c:6:47: note: each undeclared identifier is reported only once for each  
function it appears in
```

Coding Example_13: Variable MAX created after Undefining a macro

```
#define MAX 10  
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    #undef MAX // Remove the existing macro definition  
    int MAX = 100;  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

```
C:\Users\Dell>a  
Before undefining, VALUE = 10  
After undefining, VALUE = 100
```

Coding Example_14: Variable creation same as macro, without undefining the macro

```
#define MAX 10  
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    int MAX = 100;  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Pre-processing stage output:

```
int main() {  
    printf("Before undefining, VALUE = %d\n", 10);  
    int 10 = 100;  
    printf("After undefining, VALUE = %d\n", 10);  
    return 0;  
}
```

Compilation Error:

```
check.c:1:13: error: expected identifier or '(' before numeric constant  
#define MAX 10  
           ^  
check.c:5:10: note: in expansion of macro 'MAX'  
    int MAX = 100;  
        ^~~~~
```

Conditional Compilation Directives:

This allows certain parts of the code to be compiled or skipped **based on conditions** and it is useful for writing **platform-independent code**, **debugging**, or **enabling/disabling features**. Can be performed using these directives - `#ifdef`, `#ifndef`, `#if`, `#else`, `#elif`, `#else`, `#endif`. This helps in **Including/excluding features based on build settings**, **Supporting multiple versions or configurations of a program** and, **Simplifying debugging and testing different code paths**.

#if: It allows the compiler to include or exclude parts of the code **based on constant expressions or macro values**.

#elif: It allows **checking another condition if the previous #if or #elif condition was false**. It helps make the code cleaner by avoiding multiple nested `#if-#else` blocks.

#else: Used to define a **default block of code that should be compiled when none of the preceding #if or #elif conditions are true**. It ensures that some code is always compiled, even if no condition matches.

Coding Example_15: Demo of #if #else and #elif

```
#include <stdio.h>
#define VERSION 2 // Change this to 1 and 3.
int main() {
    #if VERSION == 1
        printf("You are using Version 1\n");
    #elif VERSION == 2
        printf("You are using Version 2\n");
    #else
        printf("Unknown version\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {

    printf("You are using Version 2\n");

    return 0;
}
```

Coding Example_16: Demo of #if and #else

```
#include <stdio.h>
int main() {
    #if (2 > 1)
        printf("TRUE\n");
    #else
        printf("False\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {

    printf("TRUE\n");

    return 0;
}
```

#ifdef: “if defined” directive is used to check **whether a macro has been defined** using #define. If the macro is defined, the code inside the #ifdef block is included during preprocessing for compilation. otherwise, it is skipped. It is commonly used to control code compilation for **feature toggles, debugging, and header file protection.**

#ifndef: “if not defined” directive is used to include the statements for compilation if the macro not defined using #define.

#endif: This is used to mark the end of a conditional compilation block that began with #if, #ifdef, or #ifndef. It ensures that the compiler knows where the conditional code segment ends. It is mandatory whenever #if, #ifdef, or #ifndef is used.

Coding Example_17: Demo of #ifdef and #ifndef

```
#define LEVEL 2
int main()
{
    #ifndef MAX //No parentheses for #ifdef
        printf("MAX not defined\n");
    #endif
    #ifdef LEVEL
        printf("LEVEL defined\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("MAX not defined\n");

    printf("LEVEL defined\n");
    return 0;
}
```

Coding Example_18: Demo of #ifdef and #ifndef

```
#define LEVEL 2
int main()
{
    #ifndef MAX //No parentheses for #ifdef
        printf("MAX not defined\n");
        #define MAX 10
    #endif
    #ifdef LEVEL
        #ifdef MAX
            printf("LEVEL defined and MAX defined\n");
        #endif
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {
    printf("MAX not defined\n");

    printf("LEVEL defined and MAX defined\n");
}
```

Other Directives

The language supports preprocessor directives that offer more **control over the compilation process**. These directives are typically used for **error handling, file control, custom compiler behavior, and debugging support**. They help in managing the structure of the code, controlling the flow of compilation, enhancing portability across different platforms, and improving debugging and overall maintainability of the program.

#pragma:

A compiler-specific preprocessor directive used to provide additional instructions to the compiler and allows programmers to **enable or disable certain features or optimizations**.

Syntax: `#pragma directive-name [optional-arguments]`

#pragma startup: Runs a function before main()

#pragma exit: Runs a function just before the control returns from main()

#pragma warn: Used to **enable or disable specific compiler warnings**. It helps manage the visibility of certain warnings during compilation.

NOTE: Behavior is unique to Turbo C/C++ and was commonly used in legacy DOS applications.

Coding Example_19:

```
#include <stdio.h>
#include <conio.h>
void start_func() {
    printf("This is called before main()\n");
}
void end_func() {
    printf("This is called after main()\n");
}
#pragma startup start_func
#pragma exit end_func
int main() {
    printf("Inside main function\n");
    return 0;
}
```


`#pragma warn -<letter> // Disable a specific warning`

`#pragma warn +<letter> // Enable a specific warning`

Warning Letter	Meaning
r	Return value is missing in a non-void function
s	Statement is unreachable
p	Parameter is never used
a	Assignment inside a condition (if, while)
h	Function declared but not used
c	Possible loss of precision (conversion)

Coding Example_20:

```
#include <stdio.h>
```

```
#pragma warn -r // Disable "missing return" warning
```

```
int add(int a, int b) {
```

```
    int sum = a + b;
```

```
    // Missing return statement, but no warning
```

```
}
```

```
#pragma warn +r // Re-enable "missing return" warning
```

```
int main() {
```

```
    printf("Result: %d", add(5, 10)); // Undefined behavior
```

```
    return 0;
```

```
}
```

More for you to explore -> #pragma once, #pragma GCC optimize("O3"), #pragma GCC poison printf, #line, #error

Seamless coding with the power of preprocessor directives!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #10
Portable Program Development***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Portable Program Development**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Portable Program Development(PPD) refers to **writing solution or software that can run across multiple platforms or operating systems with minimal or no changes**. It focuses on creating **platform-independent code by following standard conventions, using portable libraries, and avoiding system-specific features**. A key part of portable programming is **writing solutions for well-defined problem statements in a way that they remain functional and consistent across environments**. This requires clear problem analysis, modular design, and careful consideration of dependencies. By developing solutions with portability in mind, developers ensure that their programs are **adaptable, maintainable, and scalable across diverse systems such as Windows, Linux, macOS, and embedded platforms**.

Benefits of PPD

1. **Wider Compatibility:** Programs can run on multiple platforms (Windows, Linux, macOS, etc.) without modification, increasing their reach.
2. **Cost-Effective:** Reduces the need for separate development and maintenance teams for different platforms.
3. **Time-Saving:** Saves time in development and testing by using a single codebase for multiple systems.
4. **Ease of Maintenance:** Bugs fixed or features added in one place benefit all platforms, making updates more efficient.
5. **Scalability:** Easier to scale or migrate software to new hardware or operating systems.
6. **Enhanced Code Reusability:** Encourages modular and clean coding practices, making components reusable across projects.
7. **Future-Proofing:** Programs remain viable even if the target environment changes, as long as the standard tools/libraries remain supported.
8. **Better Collaboration:** Teams working on different platforms can collaborate more easily when the code is portable.

To achieve true portability, programs often need to adapt their behavior based on the underlying system. This is where **platform-specific macros** become useful.

Using Platform-Specific Macros

Most compilers define special macros that allow the code to **detect the platform at preprocessing time** and include or exclude system-specific sections accordingly. To support cross-platform development, these macros can be used with conditional compilation directives (like `#ifdef`) to include or exclude platform-specific code. The table below lists some of the most commonly used platform-detection macros:

Macro	Purpose / Description	Typical Platform
<code>__MINGW32__</code>	Defined when using MinGW compiler	Windows (MinGW)
<code>__WIN32</code>	Defined on 32-bit and 64-bit Windows	Windows
<code>__WIN64</code>	Defined on 64-bit Windows only	Windows
<code>__unix__</code>	Defined for Unix-like systems	Unix, Linux
<code>__linux__</code>	Specifically defined for Linux systems	Linux
<code>__APPLE__</code>	Defined when compiling on Apple/macOS	macOS
<code>__ANDROID__</code>	Defined when compiling for Android	Android
<code>__CYGWIN__</code>	Defined for Cygwin environment on Windows	Windows (Cygwin)

We typically use the following format to write platform-specific code in a portable program. This ensures that the **correct code block is compiled based on the target operating system**. This approach enhances cross-platform compatibility by allowing conditional compilation based on predefined system macros.

```
#ifdef __MINGW32__  
    // Windows-specific code  
#elif defined(__unix__)  
    // Unix/Linux-specific code  
#elif defined(__APPLE__)  
    // macOS-specific code  
#endif
```

To inspect the macros defined by the compiler for a given system or environment, we can use the **gcc -E** option, which runs only the preprocessing phase of compilation and displays the output of preprocessing on the standard output device.

Coding Example_1: Portable Code to Print OS Name

```
#include <stdio.h>
int main() {
    printf("Running on: ");

#ifdef __MINGW32__
    printf("Windows (MinGW)\n");
#endif

#ifdef _WIN32
    printf("Windows\n");
#endif

#ifdef _WIN64
    printf("Windows 64-bit\n");
#endif

#ifdef __unix__
    printf("Unix-like system\n");
#endif

#ifdef __linux__
    printf("Linux\n");
#endif

#ifdef __APPLE__
    printf("macOS\n");
#endif

#ifdef __ANDROID__
    printf("Android\n");
#endif

    return 0;
}
```

Running this code on Mingw Windows 32 system.

Preprocessing stage output is:

```
int main() {
    printf("Running on: ");

    printf("Windows (MinGW)\n");

    printf("Windows\n");
# 36 "check.c"
    return 0;
}
```

Execution output:

```
C:\Users\Dell>a
Running on: Windows (MinGW)
Windows
```

Coding Example_2 : Clearing the screen on different platforms

```
#include <stdio.h>
#include <stdlib.h>

void clearScreen() {
#ifdef _WIN32
    system("cls"); // Windows
#endif

#ifdef __unix__
    system("clear"); // Unix/Linux
#endif

#ifdef __APPLE__
    system("clear"); // macOS
#endif
}

int main() {
    printf("This message will be cleared in 3 seconds...\n");

#ifdef _WIN32
    system("timeout /T 3 > nul");
//Command to wait for three seconds
//in windows
#endif

#ifdef __unix__
    system("sleep 3");
#endif

#ifdef __APPLE__
    system("sleep 3");
#endif

    clearScreen();

    printf("Screen cleared!\n");
    return 0;
}
```

Running this code on Mingw32 Windows system. Preprocessing output:

```
void clearScreen() {
    system("cls");
# 16 "check.c"
}

int main() {
    printf("This message will be cleared in 3 seconds...\n");

    system("timeout /T 3 > nul");
# 33 "check.c"
    clearScreen();

    printf("Screen cleared!\n");
    return 0;
}
```

Execution Output: Once after you run the code, screen gets cleared in 3 seconds

```
Screen cleared!
C:\Users\De11>
```

Coding Example_3 : Portable code for Character user input and display.**Version 1: Writing a separate block of code for each OS**

```
#include<stdio.h>
int main()
{
    #ifdef __MINGW32__
        printf("Windows\n Enter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        fflush(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif

    #ifdef __unix__
        printf("UNIX\nEnter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        #include<stdio_ext.h>
        __fpurge(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif

    #ifdef __APPLE__
        printf("MAC\nEnter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        fpurge(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif
    return 0 ;
}
```

**Run on Mingw32 Windows system.
Preprocessing stage output:**

```
# 2 "check.c"
int main()
{
    printf("Windows\nEnter the character\n");
    char ch1 = getchar();
    printf("Enter another character\n");
    fflush(
        (&iob[0])
    );
    # 8 "check.c" 3
    # 8 "check.c"
    char ch2 = getchar();
    printf("You typed %c and %c\n", ch1, ch2);
    # 34 "check.c"
}
```

Execution output:

```
C:\Users\Dell>a
Windows
Enter the character
s
Enter another character
i
You typed s and i
```

Version 2: Using a parameterized macro

```
#ifndef _WIN32
    #define CLEAR_INPUT_BUFFER() fflush(stdin)
#elif __APPLE__
    #include <stdio.h>
    #define CLEAR_INPUT_BUFFER() fpurge(stdin)
#elif __unix__
    #include <stdio.h>
    #define CLEAR_INPUT_BUFFER() __fpurge(stdin)
#endif
```



```
int main() {
    char ch1, ch2;

    // First character input
    printf("Enter the first character: ");
    scanf("%c", &ch1);
    CLEAR_INPUT_BUFFER();

    // Second character input
    printf("Enter the second character: ");
    scanf("%c", &ch2);
    CLEAR_INPUT_BUFFER();

    // Display characters
    printf("You entered: '%c' and '%c'\n", ch1, ch2);

    return 0;
}
```

Preprocessing output:

```
# 16 "check.c"
int main() {
    char ch1, ch2;

    printf("Enter the first character: ");
    scanf("%c", &ch1);
    fflush(
# 22 "check.c" 3
    (&_iob[0])
# 22 "check.c"
    );

    printf("Enter the second character: ");
    scanf("%c", &ch2);
    fflush(
# 27 "check.c" 3
    (&_iob[0])
# 27 "check.c"
    );

    printf("You entered: '%c' and '%c'\n", ch1, ch2);

    return 0;
}
```

Execution output:

```
C:\Users\Dell>a
Enter the first character: s
Enter the second character: i
You entered: 's' and 'i'
```

Try it-> Explore #elif defined() to rewrite the above code more compactly !

Portability = Possibility in C!!